Simulating Radiation-Induced Faults for Fault Tolerance Analysis

Navid Shamszadeh

4 October 2026

1 Purpose and scope

Radiation-induced faults can be emulated faithfully by treating each particle strike as an event in a Poisson process, whose rate comes from the radiation environment and the device's sensitivity, and whose effect is a bit-flip or glitch in a chosen hardware state element. This document gives the physics, the statistics and the injection algorithms needed to do that on an FPGA-based fractional-order spiking reservoir computer in a hardware-in-the-loop (HiL) rig.

"Radioactive decay" here covers two sources of faults:

- **On-chip decay.** Trace uranium, thorium and polonium in packaging, solder and interconnect decay and emit alpha particles directly into the silicon. This is the classic terrestrial soft-error source.
- **Environmental radiation.** Secondary neutrons from cosmic rays at ground and flight altitude, plus trapped protons, solar particles and galactic heavy ions in space. These are not decay products of the device itself, but their arrivals obey the same statistics and their effects are emulated the same way.

The document does not cover beam-testing procedures in detail, radiation-hardening by design, or analog circuit simulation of charge collection (e.g. TCAD). Those are referenced only where they set parameters for the emulation.

2 Physical background

A fault occurs when an ionizing particle deposits enough charge near a sensitive transistor junction to flip a storage node or create a voltage spike. The quantity that matters is whether the collected charge exceeds the node's **critical charge**, Q_{crit}, which shrinks as technology nodes get smaller and supply voltages drop.

2.1 Particle sources

Source	Origin	Mechanism	Where it dominates
Alpha particles (4–9 MeV)	Decay of U-238, Th-232 chains and Po-210 in solder, mold compound, packaging	Direct ionization along a short track (tens of μm in Si)	Ground level, every chip
High-energy neutrons (> 1 MeV)	Cosmic-ray air showers	Nuclear reactions with Si produce charged recoils that ionize	Ground and flight altitude
Thermal neutrons	Moderated cosmic neutrons	Capture by B-10 releases an alpha and a Li-7 ion	Devices containing boron-10
Protons	Van Allen belts, solar events	Mostly nuclear reactions; direct ionization in very small nodes	Low Earth orbit
Heavy ions	Galactic cosmic rays, solar events	Dense direct ionization, characterized by LET	Space, accelerators

Table 1: Particle sources of single-event effects.

2.2 Linear energy transfer

Direct ionization is described by linear energy transfer (LET): energy deposited per unit path length, normalized by material density, in MeV·cm^2/mg. In silicon, about 3.6 eV creates one electron-hole pair, so an LET of 1 MeV·cm^2/mg deposits roughly 10 fC per μm of track. Comparing that deposit with Q_{crit} tells you whether a strike can upset a node.

2.3 Why decay chains matter

The uranium and thorium chains have half-lives of billions of years, so their emission rate is constant over a device's life. Po-210 is the exception: its 138-day half-life means alpha emission from freshly refined lead falls noticeably over the first year. This is one of the few cases where the time dependence of decay itself appears in a fault model (see Section 4). Packaging vendors limit the problem with low-alpha and ultra-low-alpha materials.

3 Fault taxonomy

For an SRAM-based FPGA, single-event upsets in user memory and in configuration memory are the faults worth emulating first; the others are either rarer, destructive, or better treated as slow drift (Table 2).

Effect	What happens	Persistence	Emulation model	Priority
SEU (single-event upset)	One stored bit flips	Until overwritten or scrubbed	XOR one bit in a register, BRAM word or LUTRAM	High
MBU / MCU (multiple-bit / cell upset)	Several physically adjacent bits flip from one strike	Until overwritten	Flip a cluster of neighbouring bits	Medium
SET (single-event transient)	A brief voltage pulse on combinational logic, latched only if it arrives at a clock edge	One cycle, if latched	Corrupt one signal for one cycle (e.g. spurious or missing spike)	Medium
Configuration upset	A configuration bit flips, changing routing or LUT contents	Until the frame is scrubbed or reconfigured	Flip configuration frames with the vendor's soft-error mitigation IP	High (FPGA-specific)
SEFI (functional interrupt)	Control logic upset puts the device in a stuck or reset state	Until reset	Freeze the reservoir or force its control FSM to an illegal state	Low
SEL (latchup)	Parasitic thyristor turns on; high current	Until power-cycled; may destroy	Permanent stuck-at on the affected region; power-cycle event in the log	Low
TID (total ionizing dose)	Charge trapped in oxides shifts thresholds and leakage over time	Cumulative	Slow weight drift, rising noise floor, timing slack loss	Optional
Displacement damage	Lattice defects, mainly in optoelectronics and bipolar parts	Cumulative	Usually not relevant to CMOS FPGAs	Skip

Table 2: Radiation effects and how to emulate them.

The common software abstraction for these is the **fault model triple**: *where* (target element), *what* (bit-flip, stuck-at-0/1, transient glitch), and *when* (arrival time and duration). The rest of this document is about choosing each of those three faithfully.

4 Statistics of decay and particle arrival

Every upset source in this document can be modelled as a Poisson process: strikes are independent, and the number in any interval depends only on the interval's length and the rate. What differs between sources is how the rate is computed and whether it changes over time.

4.1 The decay law

A population of N_0 radioactive nuclei with decay constant λ evolves as

$$N(t) = N_0\, e^{-\lambda t}, \qquad \lambda = \frac{\ln 2}{t_{1/2}}. \tag{1}$$

Its activity, the decay rate, is

$$A(t) = \lambda N(t) = A_0\, e^{-\lambda t}. \tag{2}$$

For U-238 ($t_{1/2} \approx 4.5{\times}10^9$ years) the activity is constant over any device lifetime, so alpha emission is a **homogeneous** Poisson process. For Po-210 ($t_{1/2} \approx 138$ days) it is **non-homogeneous**: the rate falls by half every 138 days.

4.2 Counts and inter-arrival times

With a constant rate R (events per second), the probability of k events in an interval t is

$$P(k;t) = \frac{(Rt)^k\, e^{-Rt}}{k!}. \tag{3}$$

The time between consecutive events is exponentially distributed:

$$f(\Delta t) = R\, e^{-R\Delta t}, \qquad \mathbb{E}[\Delta t] = \frac{1}{R}. \tag{4}$$

The exponential distribution is memoryless, which is exactly why the next strike time can be sampled without tracking history. The probability of at least one upset in a mission of length T is $1 - e^{-RT}$.

4.3 Time-varying rates

For Po-210, or for an orbit passing through the South Atlantic Anomaly, the rate is a function $R(t)$. The expected count over $[0, T]$ is then

$$\Lambda(T) = \int_0^T R(t)\, dt. \tag{5}$$

Such a process is sampled by **thinning**: generate candidates at the peak rate $R_{\max}$ and accept each one at time t with probability $R(t)/R_{\max}$.

4.4 From particles to bit upsets: cross-section

A device's sensitivity is its **cross-section** σ, in cm^2 per bit: the effective target area a particle must hit to cause an upset. For a flux ϕ (particles per cm^2 per second), the upset rate per bit is $\sigma\phi$. For a spectrum of energies,

$$R_{\mathrm{bit}} = \int \sigma(E)\, \phi(E)\, dE. \tag{6}$$

For heavy ions, σ is measured against LET and usually fitted with a Weibull curve, where L_0 is the threshold LET, σ_{sat} the saturation cross-section, and W, s shape parameters:

$$\sigma(L) = \sigma_{\mathrm{sat}} \left[1 - \exp\left(- \left(\frac{L - L_0}{W} \right)^s \right) \right], \qquad L > L_0. \tag{7}$$

4.5 Rates in FIT

Terrestrial soft-error rates are reported in FIT: failures per 10^9 device-hours. A device with N_{bits} sensitive bits has

$$\mathrm{FIT} = R_{\mathrm{bit}} \cdot N_{\mathrm{bits}} \cdot 3600 \cdot 10^9, \tag{8}$$

with R_{bit} in upsets per bit per second. Vendors publish FIT per megabit for their FPGA families' configuration and block RAM, which is the most practical starting number for a thesis.

5 From physical rates to injection rates

Real upset rates are far too low to observe in a lab session, so injection is **accelerated**: faults are injected at a rate many orders of magnitude above the environment's, and the results are mapped back to equivalent exposure time. This is valid as long as faults remain rare compared with the system's recovery time, so that each fault's effect is independent of the next.

5.1 Target rate

For each target memory class i (weights, membrane state, history buffer, configuration), compute an environment rate from its bit count and per-bit cross-section, then scale by one acceleration factor AF shared by all classes:

$$R_{\text{env},i} = N_{\text{bits},i} \int \sigma_i(E)\, \phi(E)\, dE, \qquad R_{\text{inj},i} = AF \cdot R_{\text{env},i}. \tag{9}$$

Keeping one AF across classes preserves their *relative* rates, which is what makes the fault mix realistic. One wall-clock second of the accelerated experiment then represents AF seconds of exposure:

$$t_{\text{env}} = AF \cdot t_{\text{wall}}. \tag{10}$$

5.2 Choosing the acceleration factor

- **Upper bound.** The mean time between injected faults, $1/R_{\text{inj}}$, should be much longer than the reservoir's recovery time. For a fractional-order network that includes the effective memory length of the Grünwald–Letnikov history (see Section 7). If faults arrive faster than the network forgets them, effects pile up and the result no longer extrapolates to the real rate.
- **Lower bound.** Each experiment run must see enough faults to give a statistically useful failure count within your available test time (see Section 9).
- **Sweep it.** Running at several AF values and checking that failures per injected fault stay constant is a direct test of the independence assumption.

5.3 Per-bit weighting

Not all bits are equally sensitive. Configuration RAM, block RAM and flip-flops have different cross-sections, published separately by FPGA vendors. Where the vendor data does not separate them, assume equal per-bit sensitivity within a memory type and say so.

5.4 Environment presets

The environment sets $\phi(E)$. The values in Table 3 are order-of-magnitude guides only (approximate, from general literature); use a radiation-environment tool for numbers you report.

Preset	Dominant particles	Relative upset rate	Tool for exact spectrum
Sea level, packaged chip	Alphas from packaging, high-energy and thermal neutrons	1 (reference)	JEDEC JESD89 reference flux
Commercial flight altitude (∼12 km)	Neutrons	Neutron flux up to ∼26,000 n/cm^2·h vs ∼13 at sea level (up to ∼2,000×)	JESD89 altitude scaling; aviation models
Low Earth orbit	Trapped protons, heavy ions; peaks in South Atlantic Anomaly	Highly orbit-dependent; time-varying	CREME96, SPENVIS, OMERE
Geostationary / interplanetary	Galactic heavy ions, solar particle events	Event-dominated; bursts of orders of magnitude	CREME96, SPENVIS, OMERE
Accelerator or beam facility	Configurable neutron, proton or ion beam	Set by the facility	Facility dosimetry

Table 3: Radiation-environment presets.

A useful thesis framing is to report every result twice: as performance versus injected faults per neuron per second, and translated into the equivalent exposure in at least one named environment.

6 Simulation algorithms

Use event-driven sampling in software and per-cycle Bernoulli sampling in hardware; both produce the same Poisson process when the per-cycle probability is small. Each injection then needs a target, a bit and a fault type.

6.1 When: event-driven sampling (software, MicroBlaze, host)

Draw exponential inter-arrival times by inverse-transform sampling, with U uniform on $(0, 1]$:

$$\Delta t = -\frac{\ln U}{R_{\text{inj}}}. \tag{11}$$

This is exact, needs one random number per fault, and is the right choice for the reference simulation and for a soft-core injector.

6.2 When: per-cycle Bernoulli sampling (on-fabric)

At every clock cycle of period T_{clk}, inject with probability

$$p = 1 - e^{-R_{\text{inj}} T_{\text{clk}}} \approx R_{\text{inj}} T_{\text{clk}}. \tag{12}$$

In hardware, compare a k-bit pseudo-random word against a threshold $\lfloor p \cdot 2^k \rfloor$. A 32-bit maximal-length LFSR is cheap, but consecutive LFSR outputs are correlated, so use a wider LFSR (e.g. 64-bit) and take non-adjacent bits, or use a small xorshift generator. Bernoulli sampling is accurate while $p \ll 1$; at 100 MHz this holds for any realistic injection rate.

6.3 Where: target and bit selection

1. Pick a memory class with probability proportional to its rate share, $R_{\text{inj},i} / \sum R_{\text{inj}}$.
2. Pick an address uniformly within that class (every bit is an equal target, unless you have per-region data).
3. Pick a bit position uniformly within the word. Do **not** bias toward MSBs: physics does not care about bit significance. Significance determines the *effect*, which is what you measure.

6.4 What: fault types

Fault type	Operation on the stored word w	Clears when
SEU bit-flip	$w \leftarrow w \oplus (1 \ll b)$	Overwritten by the design or scrubbed
MBU cluster	XOR a mask of c adjacent bits starting at b	As SEU
Stuck-at-0 / stuck-at-1	Force bit b on every read	Never (permanent) or a set duration (intermittent)
SET on a signal	Invert or force the signal for one cycle	Next cycle
Neuron death	All synaptic weights of neuron n set to 0 (your current fault)	Never

Table 4: Fault types and their operations on a stored word.

For MBUs, draw the cluster size c from a distribution that falls off quickly (a geometric distribution with most mass at 1 is a reasonable default unless vendor data says otherwise). Adjacency is **physical**, not logical: if your BRAM words are interleaved, adjacent physical cells may land in different words, which is precisely why interleaving is used as mitigation.

6.5 Reference pseudocode

```
import numpy as np

rng = np.random.default_rng(seed) # log the seed for every run
rates = {"weights": R_w, "membrane": R_v, "history": R_h, "coeffs": R_c}
R_total = sum(rates.values())
classes, probs = zip(*[(k, r / R_total) for k, r in rates.items()])

t = 0.0
while t < T_run:
    t += rng.exponential(1.0 / R_total) # when
    cls = rng.choice(classes, p=probs) # which memory
    addr = rng.integers(mem[cls].depth) # which word
    bit = rng.integers(mem[cls].width) # which bit
    size = min(rng.geometric(p_mbu), mem[cls].width - bit)
    mask = ((1 << size) - 1) << bit
    schedule.append((t, cls, addr, mask)) # sent to the injector
```

Pre-computing the schedule on the host and replaying it in hardware makes every run reproducible: the same schedule can be replayed on the software model and the FPGA to check that both behave the same.

7 Fault targets in a fractional-order SNN reservoir

The Grünwald–Letnikov (GL) history buffer is the target that makes a fractional-order reservoir different from an integer-order one: an upset there persists and keeps influencing the neuron for as long as the corrupted sample stays inside the memory window.

7.1 How the GL history carries a fault

A discretized fractional-order neuron with order α and step h updates its membrane state from a weighted sum over its past L samples:

$$v_n = h^\alpha f(v_{n-1}, I_n) - \sum_{j=1}^{L} c_j^{(\alpha)}\, v_{n-j}, \qquad c_j^{(\alpha)} = \left(1 - \frac{1+\alpha}{j}\right) c_{j-1}^{(\alpha)}, \quad c_0^{(\alpha)} = 1. \tag{13}$$

A bit-flip of size δ in stored sample v_{n-k} adds an error term of about $-c_j\delta$ to every update for which that sample is still in the window, so the error's influence falls off as the coefficient magnitudes $|c_j|$, which decay roughly as $j^{-1-\alpha}$. Smaller α means slower decay: the fault lingers longer, but each step's contribution is smaller. In an integer-order neuron ($\alpha = 1$) the same flip affects only one or two following steps. Measuring this trade-off across α is a natural central experiment.

7.2 Target inventory

Target	Typical storage	Effect of an upset	Persistence	Note
Reservoir weights	BRAM / LUTRAM	Changes network connectivity	Permanent until reloaded	Static data; scrub or reload to recover
Membrane potential v_n	Registers	One-off state perturbation; may trigger or suppress a spike	Decays with neuron dynamics	Measures intrinsic resilience of the dynamics
GL history buffer	BRAM ($L \times N$ words)	Error re-enters every update for the window length	Up to L steps	Unique to fractional order; likely the largest memory in the design
GL coefficients c_j	ROM / BRAM, shared	Changes the effective order α of every neuron that uses it	Permanent until reloaded	One upset hits all neurons: a single point of failure
Spike / event lines	Wires, FIFOs	Spurious or missing spike (SET)	One event	Model as transient
Readout weights	BRAM / registers	Directly corrupts the output	Permanent until reloaded	Decide explicitly whether to protect it
Control FSM, counters	Flip-flops	Skipped updates, hangs	Until reset	SEFI-like; detect with a watchdog
Configuration memory	FPGA config RAM	Altered logic or routing	Until scrubbed	Emulated with vendor IP (see Section 8)

Table 5: Fault targets in a fractional-order spiking reservoir.

7.3 Fixed-point considerations

Because a flip's magnitude is 2^b times the LSB weight, the same physical event can be negligible or catastrophic depending on b. Report results by bit position, and note the number format (e.g. Q4.12) of each memory. A sign-bit flip in a two's-complement weight is usually the worst case. Saturating arithmetic in the update path limits how far a flipped value can propagate, and is itself a cheap mitigation to compare against.

8 Hardware-in-the-loop injection architecture

Put the injector on the FPGA fabric and use the microcontroller or soft core as the experiment controller. That keeps every injection cycle-accurate and inside the design's clock domain, while the serial link only carries schedules, observations and results.

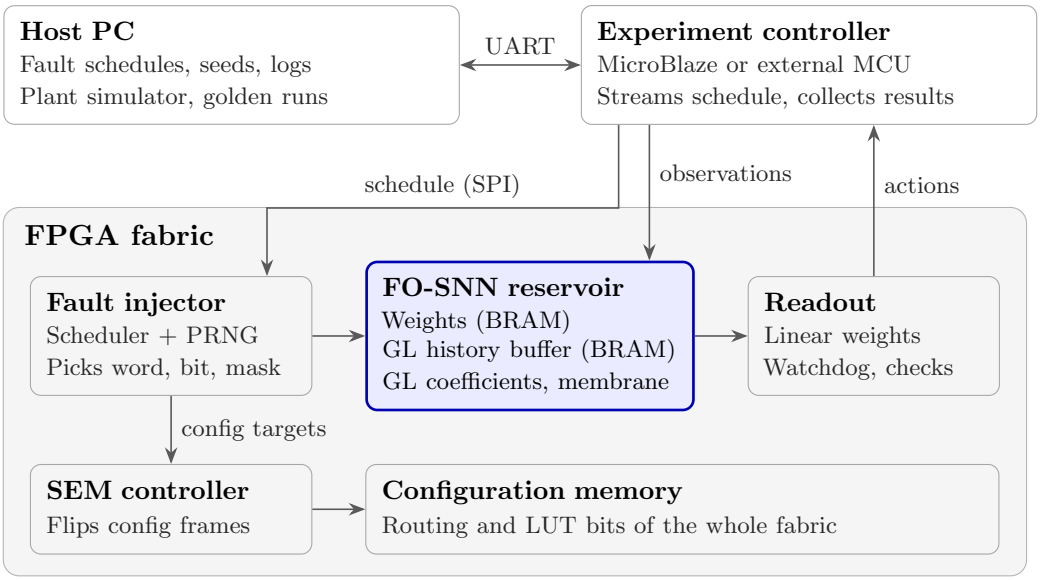


Figure 1: HiL fault-injection architecture with two injection paths. Faults reach the reservoir through user state and through configuration memory. The host plans each run and simulates the plant; the controller streams the fault schedule to the on-fabric injector and closes the control loop. User-state faults go straight into reservoir memories, and configuration faults go through the SEM controller.

8.1 Roles

- **Host PC.** Generates schedules from the rate model, stores seeds, runs the plant simulation (cart-pole, or a quadruped simulator) and the golden runs, and logs every outcome.
- **Experiment controller.** A MicroBlaze is simplest because it shares the clock and bus with the reservoir. An external MCU such as an Arduino works for low rates, but SPI latency and jitter make it unsuitable as the injector itself.
- **Fault injector.** Holds the next scheduled event (time, memory, address, mask) or a Bernoulli threshold and PRNG for free-running mode. It applies the mask by read-modify-write through the **second port** of each true dual-port BRAM, so the reservoir's datapath is untouched.
- **SEM controller.** The vendor IP for configuration-memory scrubbing, used here in its injection mode. Scrubbing between runs restores the configuration.
- **Monitors.** A watchdog on the reservoir update loop, optional parity on memories, and a fault-event log timestamped in clock cycles.

8.2 Timing rules

- Inject only between neuron update steps, or arbitrate the BRAM port, so an injection never collides with a datapath write.
- Log the exact cycle of each injection; recovery time is measured from it.
- After every run, reload weights and coefficients, clear history buffers and scrub configuration, so runs stay independent.

9 Experimental methodology and metrics

Run fault injection as a statistical campaign: many independent runs per configuration, every run's seed and schedule logged, and every reported failure probability given with a confidence interval.

9.1 Campaign structure

1. **Golden run.** Run the task with no faults to record the reference output and performance for each input sequence and seed.
2. **Single-fault runs.** Inject exactly one fault per run at a random time and location. This yields per-target vulnerability and the recovery-time distribution without interaction effects.
3. **Rate sweeps.** Inject Poisson faults at a ladder of rates (e.g. log-spaced over four decades) to get degradation curves.
4. **Factor sweeps.** Repeat steps 2–3 across fractional order α (including $\alpha = 1$), history length L, reservoir size and mitigation on/off.

9.2 Outcome classification

Compare each faulty run with its golden run and assign one class (Table 6).

Outcome	Definition (example for cart-pole)
Masked	Output and task metric indistinguishable from golden
Degraded	Task still succeeds, metric worse beyond a set tolerance
Failure (silent)	Task fails (pole falls) with no internal error flagged
Detected	A watchdog, parity or range check flags the error
Hang	Reservoir stops producing output; needs reset

Table 6: Outcome classes for a faulty run.

9.3 Metrics

- **Architectural vulnerability factor (AVF)** per target: the fraction of injected faults that cause a non-masked outcome. Effective failure rate = $R_{\text{env}} \times$ AVF.
- **Degradation curve:** task metric (e.g. NRMSE for NARMA-10, balance time for cart-pole) versus injection rate.
- **Critical rate:** the injection rate at which the metric crosses a set threshold; compare across α.
- **Recovery time:** steps from a transient fault until the reservoir state returns within ε of the golden trajectory. This is where fractional memory should show up most clearly.
- **Bit-position sensitivity:** AVF versus bit index per memory class.

9.4 How many runs

For a failure probability p estimated from n runs, the Wilson interval is a robust choice for small counts. For planning, statistical fault injection literature [6] gives a sample size for a margin of error e at confidence coefficient t ($t = 1.96$ for 95%), over a fault population of size N:

$$n = \frac{N}{1 + e^2 \dfrac{N-1}{t^2\, p(1-p)}}. \tag{14}$$

Using $p = 0.5$ gives the worst case. For a large N, 95% confidence and a 1% margin this is about 9,600 injections per configuration, which is feasible in hardware but tells you why fast on-fabric injection matters.

10 Validation and limitations

Fault emulation reproduces the *logical* effects of radiation, not the physics, so its results are only as credible as the rates and fault models fed into it. State the assumptions explicitly and validate what you can.

10.1 What emulation captures well

- Bit-flips in user-visible state (registers, BRAM, LUTRAM), with exact control of time, location and bit.
- Configuration upsets at real frame addresses, via the vendor soft-error mitigation controller, which inverts a single configuration bit by read-modify-write; multi-bit errors are built from repeated injections [1].
- Reproducibility: identical schedules can be replayed on the software model and the hardware.

10.2 What it misses

- **Hidden state.** Upsets in internal device logic that is not addressable (clock managers, configuration controller, I/O state) cause SEFIs that injection cannot reach.
- **Masked configuration bits.** Configuration bits behind LUTRAM, shift registers and dynamically reconfigured resources are masked from the controller, so they cannot be injected or scrubbed by it [2]. If your history buffer lives in LUTRAM, its configuration-level vulnerability is invisible to this method; inject into it at the user level instead.
- **Physical MBU geometry.** Without the device's physical bit map, MBU clusters are approximations.
- **Transient pulse widths.** SET duration and propagation depend on circuit-level physics; one-cycle glitches are a simplification.
- **Rate realism.** Every absolute rate depends on cross-sections you did not measure. Report results normalized to injected faults, and treat the environment translation as an estimate.

10.3 Threats to validity

Threat	Mitigation
Acceleration makes faults overlap	Sweep AF; check failures per fault stay constant
Injector perturbs timing of the design under test	Inject through dedicated ports or between update steps; compare golden runs with injector idle vs absent
Correlated pseudo-random numbers bias locations	Use a wide generator; test location histograms for uniformity
Software model and hardware disagree	Replay identical schedules on both; compare state traces bit-exactly
Too few runs for rare outcomes	Size campaigns with the sample-size formula; report intervals

Table 7: Threats to validity and mitigations.

10.4 Standards and ground truth

Terrestrial soft-error testing and reporting is defined by JEDEC JESD89, which notes that a full SER assessment needs either an unaccelerated test or accelerated data for both the alpha and cosmic-ray components [4]. A thesis does not need beam time, but citing vendor SER data measured under that standard anchors your rates in real measurements. If beam access ever becomes possible, the same HiL rig and metrics can be reused unchanged, with the injector switched off.

References

[1] AMD, *Soft Error Mitigation Controller Product Guide (PG036)*: Injection. https://docs.amd.com/r/en-US/pg036_sem/Injection. Single-bit configuration injection by read-modify-write; multi-bit by repetition.

[2] AMD, *UltraScale Architecture Soft Error Mitigation Controller Product Guide (PG187)*: Configuration Memory Masking. https://docs.amd.com/r/en-US/pg187-ultrascale-sem/Configuration-Memory-Masking. Which configuration bits are masked from injection and scrubbing.

[3] AMD, *UltraScale Architecture Soft Error Mitigation Controller Product Guide (PG187)*: General Rules for Error Injection. https://docs.amd.com/r/en-US/pg187-ultrascale-sem/General-Rules-for-Error-Injection. Valid frame address ranges.

[4] JEDEC, *JESD89B: Measurement and Reporting of Alpha Particle and Terrestrial Cosmic Ray Induced Soft Errors in Semiconductor Devices*. https://www.jedec.org/node/8374. The terrestrial SER test and reporting standard.

[5] Texas Instruments, *Soft error rate FAQs*. https://ti.com/support-quality/faqs/soft-error-rate-faqs.html. Sea-level and flight-altitude neutron flux; low-alpha materials.

[6] R. Leveugle, A. Calvez, P. Maistri and P. Vanhauwaert, "Statistical fault injection: quantified error and confidence," in *Design, Automation & Test in Europe (DATE)*, 2009. https://hal.archives-ouvertes.fr/hal-00386107. Sample-size formula used in Section 9.

[7] R. C. Baumann, "Radiation-induced soft errors in advanced semiconductor technologies," *IEEE Transactions on Device and Materials Reliability*, 2005. Survey of alpha, neutron and B-10 mechanisms. (Cited from memory; verify before use.)

[8] E. Petersen, *Single Event Effects in Aerospace*. Wiley-IEEE Press, 2011. Cross-sections, Weibull fits and rate prediction. (Cited from memory; verify before use.)

[9] CREME96, SPENVIS and OMERE: radiation-environment and SEE-rate tools for space presets.

[10] I. Podlubny, *Fractional Differential Equations*, Mathematics in Science and Engineering, vol. 198. San Diego: Academic Press, 1999. Grünwald–Letnikov discretization and the short-memory principle.